

13강

메서드





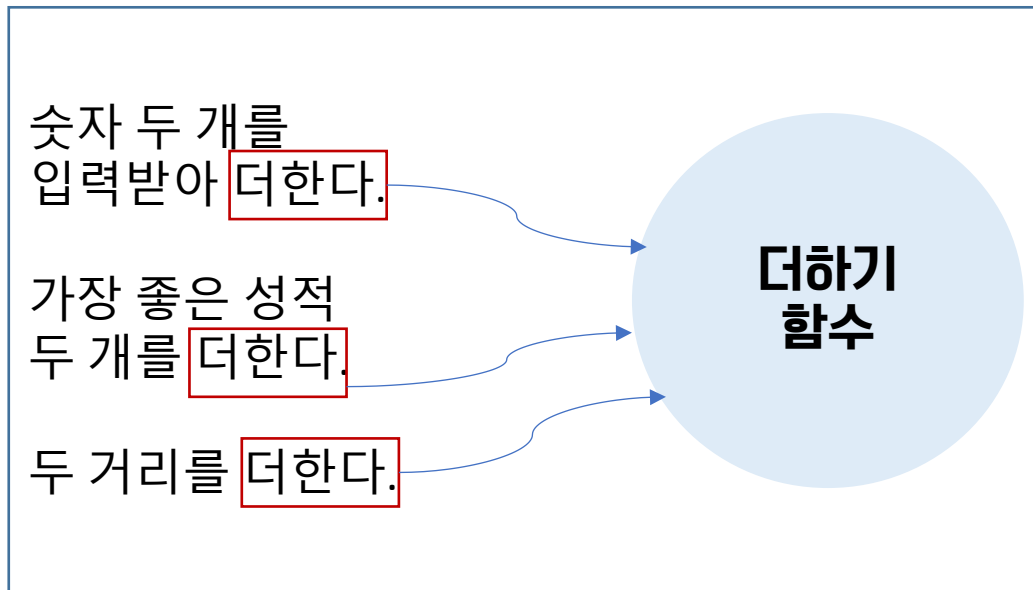
➤ 함수의 일종

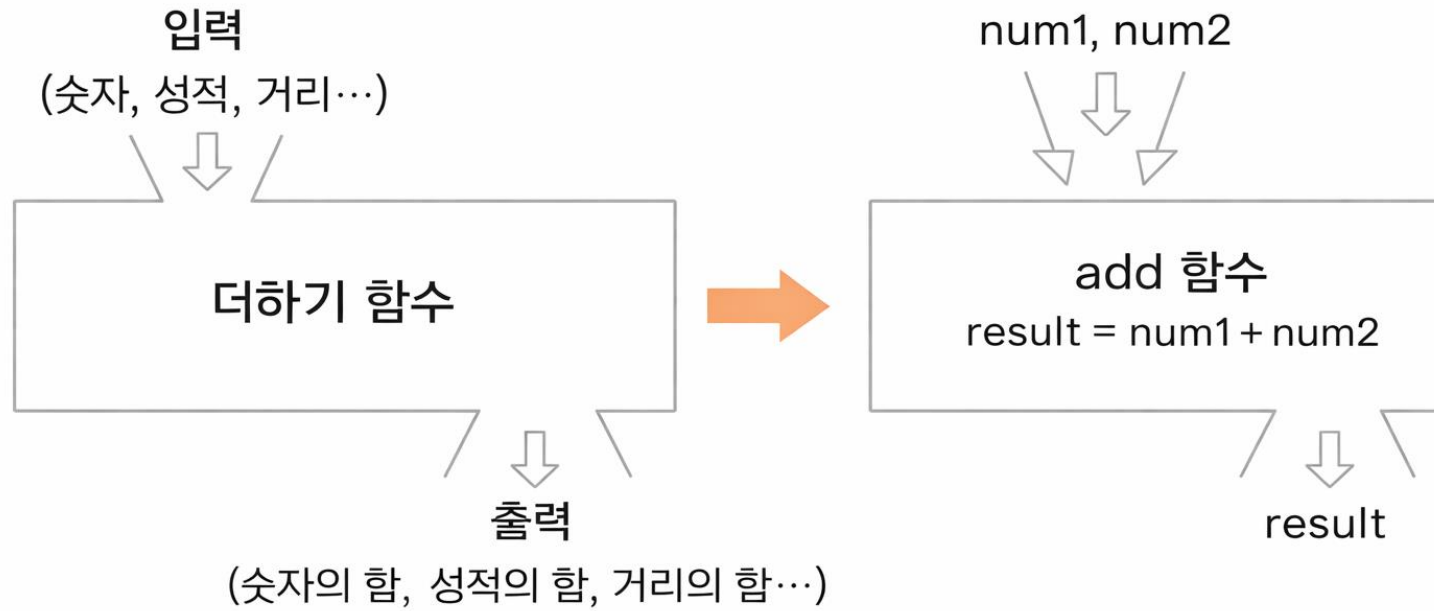
객체의 기능을 제공하기 위해 클래스 내부에 구현되는 함수

➤ 함수란 ?

하나의 기능을 수행하는 일련의 코드

중복되는 기능은 함수로 구현하여 함수를 호출하여 사용함

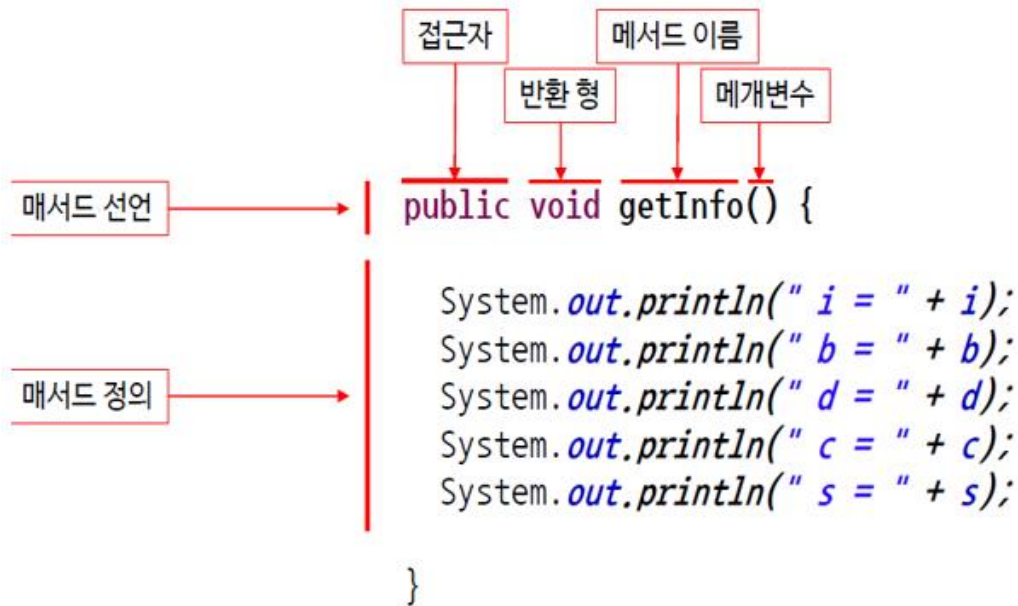






- 메서드도 변수와 같이 선언 및 정의 후 호출해서 사용한다.

메서드 선언 및 정의



메서드 호출

```
ChildClass childClass = new ChildClass();  
childClass.getInfo();
```



- 메서드를 호출할 때 데이터를 전달 할 수 있다.

```
public void getInfo() {
```

```
...
```

```
}
```

```
public void setInfo(int i, boolean b, double d, char c, String s) {
```

```
...
```

```
}
```

매개변수(parameter)는 필요시에만 정의된다.

매개변수(parameter)는 자료형과 변수(지역변수)로 이루어진다.



- 이름은 같고, 매개변수의 개수 또는 타입이 다른 메서드를 만들 수 있다.

```
public void getInfo() {  
    System.out.println("-- getInfo - I --");  
}  
  
public void getInfo(int x, int y) {  
    System.out.println("-- getInfo - II --");  
    System.out.printf("parameter --> x : %d, y : %d\n", x, y);  
}  
  
public void getInfo(String s1, String s2) {  
    System.out.println("-- getInfo - III --");  
    System.out.printf("parameter --> s1 : %s, s2 : %s\n", s1, s2);  
}
```



```
<terminated> MainClass (10) [Java Application] C:\Program  
ChildClass constructor  
-- getInfo - I --  
i = 10  
b = true  
d = 1.23  
c = j  
s = java  
-- getInfo - II --  
parameter --> x : 10, y : 20  
-- getInfo - III --  
parameter --> s1 : str1, s2 : str2
```



➤ 함수를 코드로 구현

함수의 이름, 매개변수, 반환 값을 선언하고 코드를 구현 함

```
public int add(int num1, int num2) {  
    int result;  
    result = num1 + num2;  
    return result;  
}
```

① 함수 이름 : 함수의 기능과 관련하여 명명

② 매개 변수 : 함수의 수행을 위해 필요한 변수

③ return : 함수 수행 결과를 반환하기 위한 예약어

④ 함수 반환 형 : 반환 값의 자료형을 나타냄

반환 값이 없는 경우 void라고 씀



```
package classPart;

public class FunctionTest {

    public static void main(String[] args) {

        int num1 = 10;
        int num2 = 20;

        int sum = add(num1, num2);
        System.out.println(num1 + " + " + num2 + "= " + sum + " 입니다.");
    }

    public static int add(int n1, int n2) {
        int result = n1 + n2;
        return result;
    }
}
```

add() 함수 호출

add() 함수



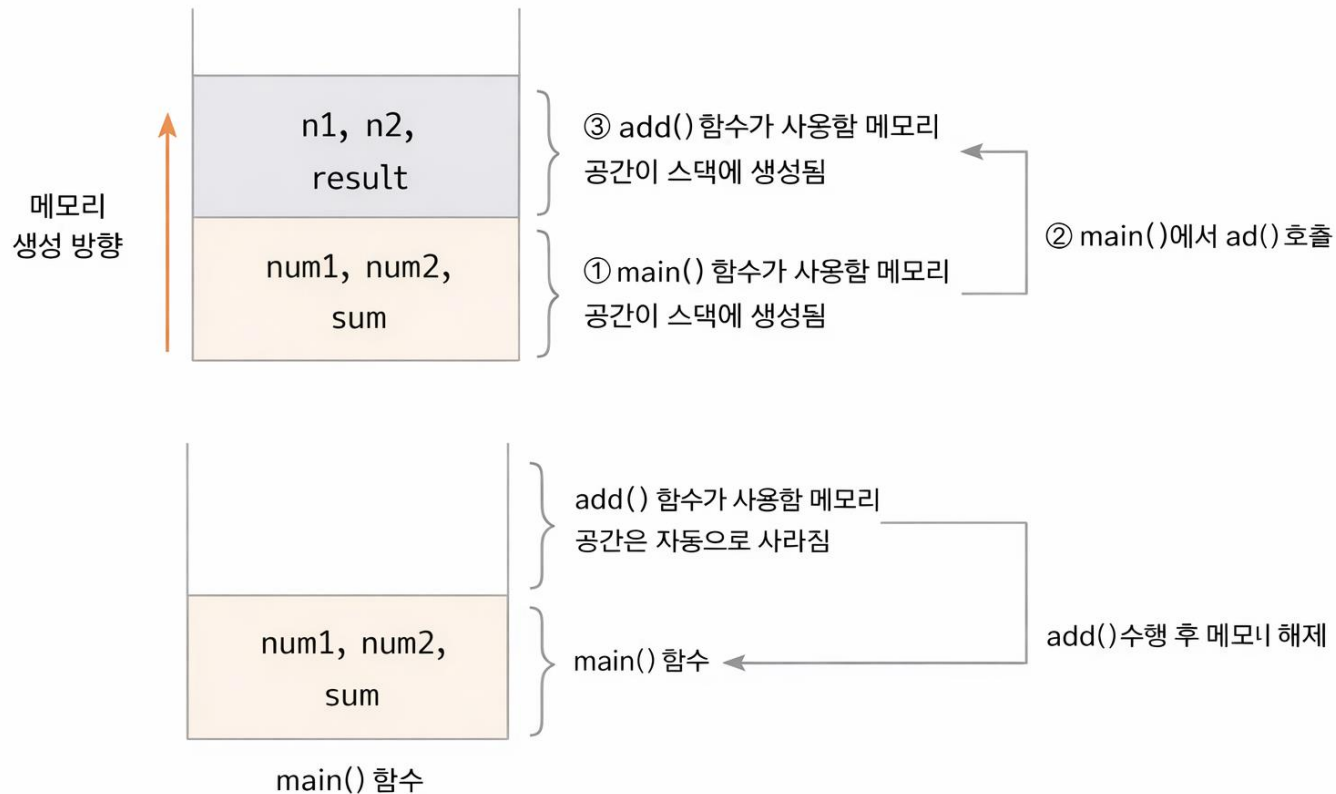
```
Console x
terminated> FunctionTest [Java Applica
10 + 20= 30 입니다.
```




➤ 함수가 호출될 때 사용하는 메모리 - 스택(stack)

함수의 기능 수행이 끝나면 자동으로 반환되는 메모리

함수 호출과 스택 메모리 구조





- 클래스의 메서드는 멤버 변수를 사용하여 기능 구현
학생의 이름을 반환하는 메서드

```
public class Student {
```

```
//멤버변수
```

```
int studentID;
```

```
String studentName;
```

```
String address;
```

```
int grade;
```

```
public String getStudentName() {
```

```
    return studentName;
```

```
}
```

```
}
```

학생의 이름을 반환하는
메서드



Java에서 **Return**은 두가지 기능을 한다.

- 첫째로 단어 그 의미대로 반환 값을 반환해주는 역할을 한다.
- 둘째로 해당 메소드를 그 즉시 종료시켜주는 역할을 한다.
- Return을 사용하려면 접근 제어자 뒤에 반환형을 적어줘야 사용이 가능하다.
- 반대로 반환형을 적어줬는데 return을 적지않거나, 리턴 할 값이 반환형과 다르다면 오류가 발생한다. 또한 반환형이 있다는 것은 그 메서드 내에서 리턴 값을 만들어내는 경우도 있겠지만, 대부분은 매개변수를 받아와서 그 매개변수를 가공하여 반환하는 경우가 일반적이다.



```
package classpart07;

public class ReturnClass {

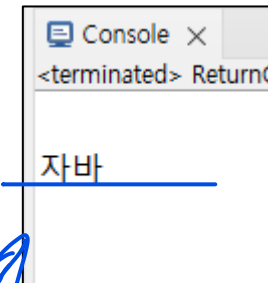
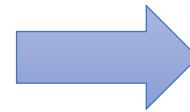
    public void Retruint() {
        int k = 10;
    }

    public String ReturnString() {
        return "자바";
    }

    public static void main(String[] args) {
        ReturnClass re = new ReturnClass();
        re.Retruint();
        String reS = re.ReturnString();
        System.out.println();
        System.out.println(reS);
    }
}
```

← 반환값

→ 호출





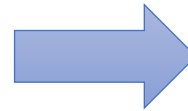
```
package classpart07;

public class Return02 {

    public int test(int[] arr) {
        int cnt = 0;
        for(int i=0; i<arr.length; i++) {
            if(arr[i] % 4 == 0) {
                cnt += 1;
            }
        }
        return cnt;
    }

    public static void main(String[] args) {
        Return02 re = new Return02();
        int[] arr = {10, 13, 14, 56, 70, 100};
        System.out.println(re.test(arr));
    }
}
```

4의 배수 개수



Console x
<terminated> Retu
2



```
package classPart;

import java.util.Scanner;

public class ReturnClass {

    int num1 = 10;
    int num2 = 20;

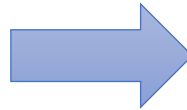
    public void test02() {
        if(num1 >= num2) {
            System.out.println("크다");
        }
        return;
    }

    public static void main(String[] args) {

        Scanner scan = new Scanner(System.in);
        ReturnClass r = new ReturnClass();

        while(true) {
            System.out.println("메뉴출력");
            int sel = scan.nextInt();
            if(sel == 1) {
                r.test02();
            } else {
                System.out.println("종료");
                return;
            }
        }
    }
}
```

- void로 선언된 모든 메서드는 값을 반환하지 않습니다.
- return 문을 포함할 필요는 없지만 포함할 수 있습니다
 - 이 경우 return 문을 사용하여 제어 흐름 블록에서 분기하고 메서드를 종료할 수 있으며 다음과 같이 간단히 사용된다.



```
Console x
<terminated> Ret
메뉴출력
1
메뉴출력
1
메뉴출력
2
종료
```

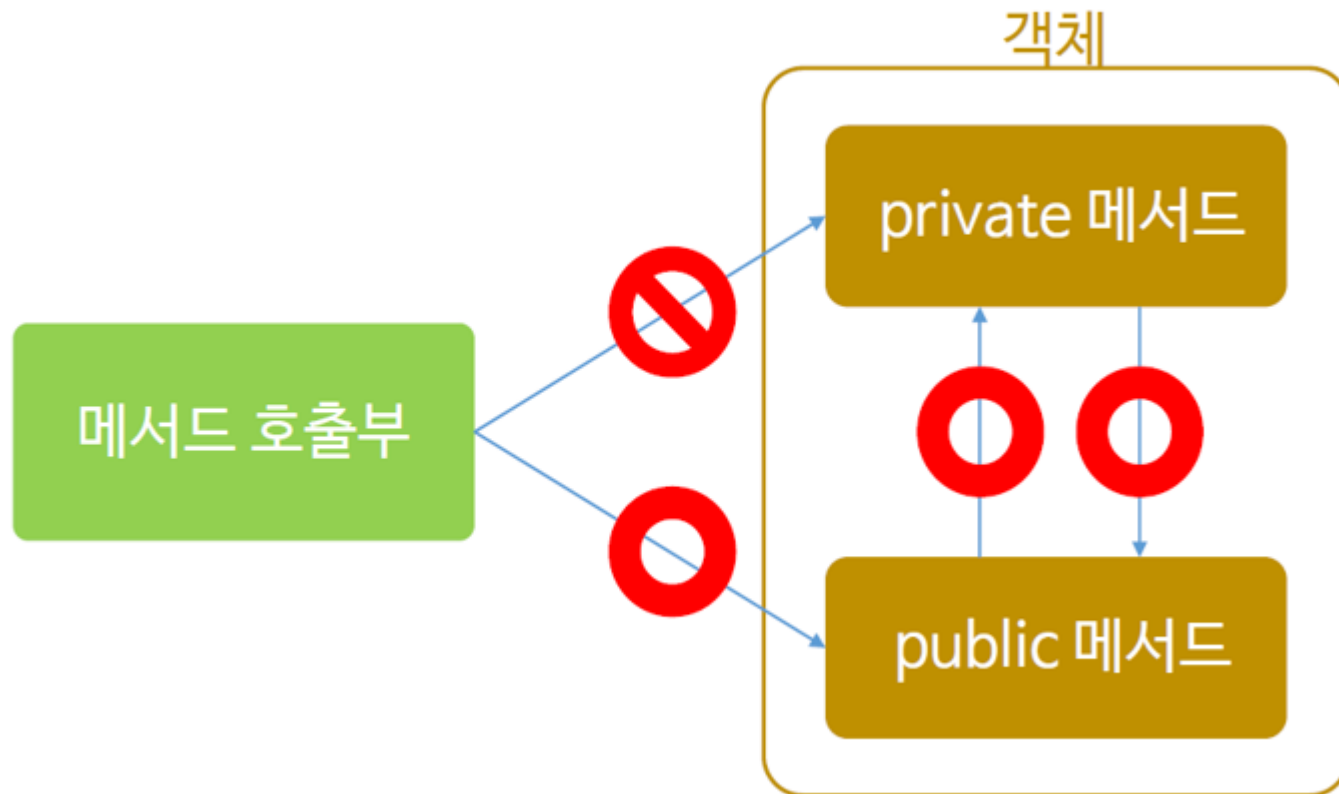


- public : 모든 접근을 허용
- protected : 같은 패키지(폴더)에 있는 객체와 상속관계의 객체들만 허용
- default : 같은 패키지(폴더)에 있는 객체들만 허용
- private : 현재 객체 내에서만 허용

접근 제어자	외부 패키지에서 사용	상속 관계에서 사용	외부 클래스에서 사용	같은 클래스에서 사용
Public	○	○	○	○
Protected	X	○	○	○
Default	X	X	○	○
private	X	X	X	○



- 메서드를 호출할 때 접근 제어자에 따라 호출이 불가 할 수 있다.



연습문제





Package 명 classpart06로 작성 할 것

Class 명 : ClassEx06, MainClass 두개의 클래스를 작성 할 것

문제1>

OMR카드 : 클래스 + 변수

1. 배열 answer는 시험문제의 정답지이다.
2. 배열 hgd에 1~5 사이의 랜덤 숫자 5개를 저장한다.
3. answer와 hgd 값을 비교해 정오표를 출력한다.
4. 한 문제당 20점이다.

예)

answer = 1 3 4 2 5

hgd = 1 3 2 1 4

정오표 = ○ ○ x x x

성적 : 40



Package 명 classpart07로 작성 할 것

Class 명 : ClassEx07, MainClass 두개의 클래스를 작성 할 것

문제2>

영수증 출력하기 : 클래스 + 변수

1. 햄버거 주문을 받아 영수증을 출력한다.
2. 출력 내용은 각 메뉴의 주문 수량과 총 금액 및 잔돈을 표시한다.
3. 메서드 분리까지 고려하시오.

```
int[] arPrice = { 2500, 3800, 1500, 1000};  
String[] arMenu = {"치즈버거", "불고기버거", "감자튀김", "콜 라"};
```

```
int[] arCount = new int[4]; //각 메뉴별 개수
```

```
int total = 0;
```



[맘스터치 햄버거]

- 1. 치즈버거 2500원
- 2. 불고기버거 3800원
- 3. 감자튀김 1500원
- 4. 콜 라 1000원

5. 주문하기

메뉴 선택 : 1

[맘스터치 햄버거]

- 1. 치즈버거 2500원
- 2. 불고기버거 3800원
- 3. 감자튀김 1500원
- 4. 콜 라 1000원

5. 주문하기

메뉴 선택 : 4

[맘스터치 햄버거]

- 1. 치즈버거 2500원
- 2. 불고기버거 3800원
- 3. 감자튀김 1500원
- 4. 콜 라 1000원

5. 주문하기

메뉴 선택 : |

메뉴 선택 : 5

총 금액은 3500원 입니다. 돈을 입력하세요 : 4000

맘스터치

ITEM	QTY	AMT
치즈버거	1	2500
불고기버거	0	0
감자튀김	0	0
콜 라	1	1000

합계금액	3500
받은금액	4000

잔	돈	500
---	---	-----